
Vasicek Short-Rate Model and Swaption Valuation

Capstone Project Report

Indian Institute of Quantitative Finance

Author: Hisay Lama

Programme: Certificate in Financial Engineering (CPFE)

Date October 4, 2026

Abstract

This project report presents a complete implementation of the Vasicek one-factor short-rate model, calibrated to recent Federal Reserve rate data. Using the calibrated parameters, we derive analytical zero-coupon bond (ZCB) prices via the exponential-affine formula, replicate those prices via Monte Carlo simulation with antithetic variates, compute the swap rate for a 5-year interest rate swap, and price a European call option on a ZCB. Furthermore, we construct a SOFR-based discount curve via linear interpolation and apply Black's formula to value a European swaption, with discussion of the appropriate measure-theoretic framework (numeraire) for both European and Bermudan swaptions. All models are implemented in Python with modular, production-oriented code. The corresponding Python source code is hosted in the GitHub repository linked within this report.

Contents

1	Project Statement	2
2	Vasicek Short-Rate Model (Part I)	4
2.0.1	Stochastic Differential Equation	4
2.0.2	Analytical Solution: Continuous and discrete representation	4
2.1	Calibration to Federal Reserve Rate Data (Part I a)	6
2.1.1	Evaluation of a and b	6
2.1.2	Data and Implementation	6
2.1.3	Calibration Results	7
2.2	Term Structure and ZCB Pricing (Parts Ib)	8
2.2.1	Exponential-Affine Formula	8
2.3	Monte Carlo Simulation and Bond Option Pricing (Part Ic)	11
2.3.1	ZCB Price via Monte Carlo (Part Ic)	11
2.3.2	Variance Reduction: Antithetic Variates	13
2.4	Swap Rate (Part Id)	15
2.5	European Call Option on a ZCB (Part Ie)	16
3	Swaption Pricing (Part II)	18
3.1	SOFR (Secured Overnight Financing Rate) Yield Curve (Part IIa)	18
3.2	European Swaption Valuation (Part IIb)	20
3.2.1	Swaption Specification	20
3.2.2	Annuity Factor and Forward Swap Rate	21
3.2.3	Black's Formula	21
3.2.4	Result	22
3.2.5	Implementation	22
3.3	Numerator Theory (Part IIc and IId)	24
3.3.1	The Concept of Numeraire	24
3.3.2	Numerator for the European Swaption	24
3.3.3	Numerator for the Bermudan Swaption	24
4	Code Repository Structure	26

1 Project Statement

PART I

You are using Vasicek model as the short rate process that is calibrated as:

$$dr(t) = a(b - r(t)) dt + \sigma dZ(t)$$

- Use the FRED Rates recent data and calculate the appropriate value of a and b . Write a generic code in Python, that reads the data from a link and generates the values at any point.
- Use the above model to calculate the analytical price of a 5-year Zero-coupon (with a face value of \$1) bond.
- Using Monte Carlo simulation technique, calculate the above 5-year Zero-coupon bond price assuming that instantaneous short rate $r(0) = 0.04$. List down variance reduction techniques for Monte Carlo Simulation and explain any one of them.
- Calculate the swap rate (fixed rate leg) of an interest rate swap that has a maturity of 5 years and the floating leg has an annual coupon payment frequency linked to SOFR.

[Hint:] Note the price of a swap is given as:

$$\text{Swap Rate} = \frac{1 - Z(N)}{\sum_{i=1}^N Z(i)}$$

Where $Z(i)$ is the zero-coupon price for maturity i .

- Calculate the price of a European Call option on 5-year zero-coupon bond (with a face value of \$1000) and a maturity of 4 years and Strike price of \$900 using the Vasicek model that has been calibrated as above.

Note this solution allows for simulation of negative interest rates that is a known limitation of Vasicek model.

[Hint:] The payoff of call option is:

$$\max(Z[t, T] - K, 0)$$

and you may ignore the discounting of payoff or use the 4-year zero spot rate to discount the payoff.

And also note that $Z[t = 4y, T = 5y]$ can be computed using exponential affine formula where the $t = 4y$ and $r(t = 4y)$ can be simulated using same steps as Part b above.

PART II

Value a European Swaption that gives the holder right to enter into a 5-year swap after 2 years. The swap rate fixed is 4.5% and will receive floating rate. Cash flows on swap are going to be exchanged semi-annually. The volatility of forward swap rate is 15%. Notional is \$100 and compounding is continuous.

- (a) Prepare the yield curve from SOFR rate for discounting. Use linear interpolation approach to get the yield curve.
- (b) Use the discount curve of (a) to value the European Swaption as mentioned.
- (c) Explain the concept of numeraire and suggest a suitable numeraire to solve the valuation of a European Swaption.
- (d) Suggest a similar numeraire for a Bermudan swaption and explain the valuation.

2 Vasicek Short-Rate Model (Part I)

The Vasicek model is a one-factor short-rate model that describes the stochastic evolution of the instantaneous interest rate driven by a single Wiener process under a risk-neutral measure. It is widely used for interest-rate derivative valuation and has also been adapted to credit-risk modelling.

2.0.1 Stochastic Differential Equation

The Vasicek model specifies the instantaneous short rate $r(t)$ under the risk-neutral measure as:

$$dr(t) = a(b - r(t)) dt + \sigma dW(t), \quad (1)$$

where:

- a is the **mean-reversion speed** (units: 1/year)
- b is the **long-run mean** (risk-neutral) of the short rate
- σ is the **instantaneous volatility**
- $W(t)$ is a standard Brownian motion

The key feature of Eq (1) is the *mean-reverting drift*: when $r > b$ the drift is negative, pulling the rate down, and vice versa. This is qualitatively consistent with the empirical behaviour of central bank policy rates.

2.0.2 Analytical Solution: Continuous and discrete representation

Continuous Time Representation. To solve the SDE Eq. (1), we multiply both sides by the integrating factor e^{at} . Since e^{at} is deterministic, Itô's product rule gives no second-order term, and

$$d(e^{at}r(t)) = ae^{at}r(t) dt + e^{at} dr(t) = abe^{at} dt + \sigma e^{at} dW(t). \quad (2)$$

Integrating over $[s, t]$ yields

$$e^{at}r(t) - e^{as}r(s) = b(e^{at} - e^{as}) + \sigma \int_s^t e^{au} dW(u), \quad (3)$$

and multiplying by e^{-at} gives the exact solution

$$r(t) = r(s)e^{-a(t-s)} + b(1 - e^{-a(t-s)}) + \sigma \int_s^t e^{-a(t-u)} dW(u), \quad t \geq s. \quad (4)$$

The first two terms in Eq (4) are deterministic given $r(s)$, while the third term constitutes the stochastic integral, characterised by the following standard result.

$$\mathbb{E}[r(t) \mid r(s)] = r(s)e^{-a(t-s)} + b(1 - e^{-a(t-s)}), \quad (5)$$

$$\text{Var}[r(t) \mid r(s)] = \frac{\sigma^2}{2a}(1 - e^{-2a(t-s)}). \quad (6)$$

Mean-Reversion Half-Life: The half-life of a rate deviation from b is expressed as

$$t_{1/2} = \frac{\ln 2}{a}. \quad (7)$$

For example, $a = 0.30$ implies $t_{1/2} \approx 2.3$ years, meaning deviations from the long-run mean decay to half their initial size over roughly 2.3 years.

Discrete Time Representation. The continuous analytical solution in Eq. (4) can be discretised over a time step Δt . Setting $s = t$ and replacing t with $t + \Delta t$ in Eq. (4) yields:

$$r_{t+\Delta t} = r_t e^{-a\Delta t} + b(1 - e^{-a\Delta t}) + \epsilon_t, \quad \epsilon_t = \sigma \int_t^{t+\Delta t} e^{-a(t+\Delta t-u)} dW(u). \quad (8)$$

The third term ϵ_t in Eq (8) satisfy

$$\epsilon_t \sim \mathcal{N}\left(0, \frac{\sigma^2}{2a} (1 - e^{-2a\Delta t})\right), \quad (9)$$

are i.i.d. across non-overlapping time steps, and are independent of r_t . Note that Eq.(9) is derived from Lemma 1.

Lemma 1: Let $W(u)$ be a standard Brownian motion and let $f : [t_0, t_1] \rightarrow \mathbb{R}$ be a deterministic function with $\int_{t_0}^{t_1} f(u)^2 du < \infty$. Then

$$\int_{t_0}^{t_1} f(u) dW(u) \sim \mathcal{N}\left(0, \int_{t_0}^{t_1} f(u)^2 du\right). \quad (10)$$

Proof: The integral is the limit of sums $\sum_k f(u_k) [W(u_{k+1}) - W(u_k)]$. The Brownian increments are independent and Gaussian with mean zero and variance $u_{k+1} - u_k$, and the weights $f(u_k)$ are non-random. Each sum is therefore Gaussian with mean zero and variance $\sum_k f(u_k)^2 (u_{k+1} - u_k)$. A limit of Gaussian random variables with converging variances is Gaussian, and the variance converges to $\int f(u)^2 du$ (the Itô isometry).

- (a) Use the FRED Rates recent data and calculate the appropriate value of a and b . Write a generic code in Python, that reads the data from a link and generates the values at any point.

2.1 Calibration to Federal Reserve Rate Data (Part I a)

To estimate the Vasicek model parameters from the Federal Funds Rate data, we fit the observed interest-rate series to the discrete-time representation of the Vasicek model. Starting from the exact solution of the Vasicek stochastic differential equation, the transition over a time interval Δt is given by:

$$r_{t+\Delta t} = r_t e^{-a\Delta t} + b(1 - e^{-a\Delta t}) + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}\left(0, \frac{\sigma^2}{2a}(1 - e^{-2a\Delta t})\right). \quad (11)$$

Setting $\alpha = e^{-a\Delta t}$, $\beta = b(1 - e^{-a\Delta t})$, and $s^2 = \frac{\sigma^2}{2a}(1 - e^{-2a\Delta t})$, this becomes:

$$r_{t+\Delta t} = \alpha r_t + \beta + \epsilon_t \quad (12)$$

this matches the standard **First-Order Autoregressive AR(1)** linear regression model with an intercept. Here, parameters α and β can be estimated by ordinary least squares (OLS) using the historical interest-rate observations.

2.1.1 Evaluation of a and b

Running OLS on the historical series $\{r_t\}$ provides estimates $\hat{\alpha}$, $\hat{\beta}$, and $\hat{s}$ of the AR(1) parameters. Here, the notation $(\hat{\alpha}, \hat{\beta}, \hat{a}, \hat{b}, \text{ and } \hat{\sigma})$ denotes the corresponding parameter estimates obtained from the observed data. The corresponding estimates of the structural Vasicek parameters are:

$$\hat{a} = -\frac{\ln \hat{\alpha}}{\Delta t}, \quad \hat{b} = \frac{\hat{\beta}}{1 - \hat{\alpha}}, \quad \hat{\sigma} = \hat{s} \sqrt{\frac{2\hat{a}}{1 - e^{-2\hat{a}\Delta t}}}. \quad (13)$$

Once $\hat{\alpha}$, $\hat{\beta}$, and the residual variance $\hat{s}^2$ have been obtained, the structural Vasicek parameters can be recovered through the inverse transformations.

2.1.2 Data and Implementation

Data are sourced from the FRED API (series DFF for the daily Effective Federal Funds Rate). The Python function below reads the data generically from any FRED URL:

```

1 import numpy as np
2 import pandas as pd
3 from scipy import stats
4
5 FRED_URL = "https://fred.stlouisfed.org/graph/fredgraph.csv?id={series}"
6
7
8 def fetch_fed_rates(series: str = "DFF",
9                     start: str = "2000-01-01",
10                    end: str | None = None) -> pd.Series:
11     """Read a FRED rate series and return it as a decimal rate."""
12     df = pd.read_csv(FRED_URL.format(series=series),
```

```

13         index_col=0, parse_dates=True)
14     df.columns = ["rate"]
15     df["rate"] = pd.to_numeric(df["rate"], errors="coerce") # FRED uses
16     df = df.loc[start:end].dropna()
17     return df["rate"] / 100 # percent
18     -> decimal
19
20 def calibrate_vasicek(rates: pd.Series, dt: float = 1/365) -> dict:
21     """
22     OLS calibration of Vasicek parameters using the exact AR(1) form
23      $r_{t+dt} = \alpha * r_t + \beta + \epsilon$ ,  $\epsilon \sim N(0, s^2)$ .
24     Returns a dict with keys: a, b, sigma.
25     """
26     r = rates.values
27     X, Y = r[:-1], r[1:] #  $r_t$  and  $r_{t+dt}$ 
28
29     # OLS regression:  $Y = \alpha * X + \beta + \epsilon$ 
30     alpha, beta, *_ = stats.linregress(X, Y)
31     resid = Y - (alpha * X + beta)
32     s2 = np.var(resid, ddof=2) # residual variance
33
34     if not 0 < alpha < 1:
35         raise ValueError(f"alpha = {alpha:.6f}: no mean reversion in
36             this window.")
37
38     # Map AR(1) coefficients back to Vasicek parameters
39     a = -np.log(alpha) / dt
40     b = beta / (1 - alpha)
41     sigma = np.sqrt(2 * a * s2 / (1 - alpha**2))
42
43     return {"a": a, "b": b, "sigma": sigma}

```

Listing 1: Vasicek calibration from FRED data (calibration.py)

2.1.3 Calibration Results

Table 1: Indicative Vasicek calibration results (2000–2024, daily DFF data)

Parameter	Symbol	Typical value	Interpretation
Mean-reversion speed	a	0.19	Half-life ≈ 3.65 years
Long-run mean	b	0.02	Risk-neutral equilibrium rate
Volatility	σ	0.010	Annualised rate vol

The mean-reversion speed $a \approx 0.19$ implies that the short rate reverts to its long-run mean with a half-life of approximately 3.65 years, broadly consistent with observed FRED rate cycles. For more details, check `01_calibration.ipynb`.

- (b) Use the above model to calculate the analytical price of a 5-year Zero-coupon (with a face value of \$1) bond.

2.2 Term Structure and ZCB Pricing (Parts Ib)

2.2.1 Exponential-Affine Formula

Under the Vasicek model, the price at time t of a bond paying \$1 at time T is:

$$Z(t, T) = A(t, T) \exp(-B(t, T) r(t)), \quad (14)$$

where the functions A and B are given by:

$$B(t, T) = \frac{1 - e^{-a(T-t)}}{a}, \quad (15)$$

$$A(t, T) = \exp \left[\left(b - \frac{\sigma^2}{2a^2} \right) (B(t, T) - (T - t)) - \frac{\sigma^2}{4a} B(t, T)^2 \right]. \quad (16)$$

We price today, so $t = 0$, and the bond matures in 5 years, so $T = 5$:

$$Z(0, 5) = A(0, 5) \exp(-B(0, 5) r(0)), \quad (17)$$

Here, parameters a , b and σ are obtained by fitting the Vasicek model to the FRED rate data in part I(a), and $r(0)$ is taken as the latest observed rate in the dataset. We first compute $B(0, 5)$, then use it to compute $A(0, 5)$, and finally obtain the bond price as $Z(0, 5) = A(0, 5) e^{-B(0, 5) r(0)}$. The function $B(t, T)$ measures the bond price's sensitivity to the current short rate $[\partial \ln Z(t, T) / \partial r(t) = -B(t, T)]$.

```

1  """
2  Vasicek model: (a) fit a, b, sigma to FRED rate data, (b) price a zero-
    coupon bond.
3
4  Model:  dr = a(b - r) dt + sigma dZ
5  Fit:    exact AR(1) form  r_{t+dt} = alpha r_t + beta + eps
6  Price:  P(t,T) = A(t,T) exp(-B(t,T) r(t))
7  """
8  import numpy as np
9  import pandas as pd
10
11  FRED_URL = "https://fred.stlouisfed.org/graph/fredgraph.csv?id={series}"
12
13
14  # ----- (a) data
    and fit
15  def load_rates(source, start=None, end=None, percent=True):
16      """Read a 2-column CSV (date, rate) from a URL or file. Returns
        decimals."""
17      df = pd.read_csv(source).iloc[:, :2]
18      df.columns = ["date", "rate"]
19      df["date"] = pd.to_datetime(df["date"])
20      df["rate"] = pd.to_numeric(df["rate"], errors="coerce") # FRED
        uses ','
21      r = df.dropna().set_index("date").sort_index()["rate"]

```

```

22     if start:
23         r = r.loc[start:]
24     if end:
25         r = r.loc[:end]
26     return r / 100.0 if percent else r
27
28
29 def infer_dt(r):
30     """Time step in years, from the spacing of the dates."""
31     gaps = np.diff(r.index.values).astype("timedelta64[D]").astype(float)
32     med = np.median(gaps)
33     if med <= 1.5:                                     # daily
34         return 1 / 252 if np.mean(gaps > 1.5) > 0.1 else 1 / 365
35     if med <= 8:
36         return 1 / 52                                 # weekly
37     if med <= 32:
38         return 1 / 12                                 # monthly
39     return 1 / 4                                       # quarterly
40
41
42 def fit_vasicek(r, dt):
43     """Fit Vasicek parameters using the exact AR(1) discretisation."""
44     x, y = np.asarray(r)[: -1], np.asarray(r)[1:]      #  $r_t$  and  $r_{t+dt}$ 
45     X = np.column_stack([x, np.ones_like(x)])
46     (alpha, beta), *_ = np.linalg.lstsq(X, y, rcond=None) # regression
47     s2 = (y - (alpha * x + beta)).var(ddof=2)           # residual
48                                                         variance
49     if not 0 < alpha < 1:
50         raise ValueError(f"alpha = {alpha:.6f}: no mean reversion in
51                             this window.")
52
53     a = -np.log(alpha) / dt
54     b = beta / (1 - alpha)
55     sigma = np.sqrt(2 * a * s2 / (1 - alpha**2))
56     return dict(a=a, b=b, sigma=sigma)
57
58 # ----- (b) bond
59     price
60 def B(t: float, T: float, a: float) -> float:
61     """Duration sensitivity  $B(t, T) = (1 - \exp(-a(T-t))) / a$ """
62     tau = T - t
63     return (1 - np.exp(-a * tau)) / a
64
65 def A(t: float, T: float, a: float, b: float, sigma: float) -> float:
66     """Factor  $A(t, T)$  capturing the long-run level  $b$  and volatility  $\sigma$ 
67         ."""
68     tau = T - t
69     B_tT = B(t, T, a)
70     exponent = (
71         (b - sigma**2 / (2 * a**2)) * (B_tT - tau)
72         - sigma**2 / (4 * a) * B_tT**2
73     )
74     return np.exp(exponent)

```

```

74
75
76 def zcb_price(r_t: float, t: float, T: float,
77             a: float, b: float, sigma: float,
78             face_value: float = 1.0) -> float:
79     """Analytical zero-coupon bond price  $P(t,T)$  under the Vasicek model.
80     """
81     return face_value * A(t, T, a, b, sigma) * np.exp(-B(t, T, a) * r_t)
82
83 # ----- run
84 if __name__ == "__main__":
85     # Part (a): fit a, b, sigma to FRED rate data
86     r = load_rates(FRED_URL.format(series="DFF"))
87     fit = fit_vasicek(r, infer_dt(r))
88     a, b, sigma = fit["a"], fit["b"], fit["sigma"]
89     r0 = r.iloc[-1]
90
91     # Part (b): 5-year zero-coupon bond, face value $1
92     t, T = 0.0, 5.0
93     print(f>Data up to {r.index[-1].date()}")
94     print(f"a = {a:.4f}, b = {b:.4f}, sigma = {sigma:.4f}, r(0) = {r0:.4f}")
95     print(f"B(0,5) = {B(t, T, a):.4f}")
96     print(f"A(0,5) = {A(t, T, a, b, sigma):.4f}")
97     print(f"P(0,5) = {zcb_price(r0, t, T, a, b, sigma):.4f}")

```

Listing 2: ZCB pricing functions (vasicek.py)

Table 2: Illustrative ZCB prices $Z(0, T)$ and continuously-compounded zero rates $y(T) = -\ln Z(0, T)/T$, for $r(0) = 0.04$ and indicative parameters $a = 0.25$, $b = 0.04$, $\sigma = 0.015$.

Maturity T (yr)	ZCB price $Z(0, T)$	Zero rate $y(T)$
1	0.962500	3.8221%
2	0.929386	3.6615%
3	0.899871	3.5168%
4	0.873315	3.3865%
5	0.849201	3.2692%

For more details, see `02_term_structure.ipynb`.

- (c) Using the Monte Carlo simulation technique, calculate the above 5-year Zero-coupon bond price assuming that the instantaneous short rate $r(0) = 0.04$. List variance reduction techniques for Monte Carlo Simulation and explain any one of them.

2.3 Monte Carlo Simulation and Bond Option Pricing (Part Ic)

2.3.1 ZCB Price via Monte Carlo (Part Ic)

The price of the zero-coupon bond is the expected discount factor over all possible paths of the short rate under the risk-neutral measure:

$$Z(0, T) = \mathbb{E} \left[\exp \left(- \int_0^T r(s) ds \right) \right], \quad T = 5, r(0) = 0.04. \quad (18)$$

Here, $\int_0^T r(s) ds$ is the total interest accumulated along one path of the short rate, and the exponential converts it into a discount factor. Since the future path of the rate is uncertain, the bond price is obtained by averaging this discount factor over all possible paths. Monte Carlo simulation estimates this average by generating a large number of simulated rate paths and taking the mean of their discount factors.

To simulate the paths, the time interval $[0, 5]$ is divided into $N = 1260$ equal steps of size $\Delta t = 5/1260$, corresponding to one step per trading day (days when markets are open). Starting from $r(0) = 0.04$, each path is generated using the exact discretisation of the Vasicek model,

$$r_{k+1} = \alpha r_k + \beta + s W_k, \quad W_k \sim \mathcal{N}(0, 1), \quad (19)$$

where $\alpha = e^{-a\Delta t}$, $\beta = b(1 - \alpha)$ and $s^2 = \frac{\sigma^2}{2a}(1 - \alpha^2)$. Because this scheme follows directly from the analytical solution of the SDE, the simulated rates carry no discretisation error at the grid points.

For each path i , the accumulated interest $\int_0^5 r^{(i)}(u) du$ is approximated by the trapezoidal rule, $\sum_k \frac{1}{2} (r_k^{(i)} + r_{k+1}^{(i)}) \Delta t$, and the corresponding discount factor is

$$D_i = \exp \left(- \int_0^5 r^{(i)}(u) du \right).$$

The bond price is then estimated as the sample mean over $M = 100,000$ paths,

$$\hat{Z}(0, 5) = \frac{1}{M} \sum_{i=1}^M D_i,$$

with standard error $\hat{\sigma}_D / \sqrt{M}$, where $\hat{\sigma}_D$ is the sample standard deviation of the discount factors.

```
1 import numpy as np
2 import pandas as pd
3 from scipy import stats
```

```

4
5 FRED_URL = "https://fred.stlouisfed.org/graph/fredgraph.csv?id={series}"
6
7
8 def fetch_fed_rates(series: str = "DFF",
9                     start: str = "2000-01-01",
10                    end: str | None = None) -> pd.Series:
11     """Read a FRED rate series and return it as a decimal rate."""
12     df = pd.read_csv(FRED_URL.format(series=series),
13                    index_col=0, parse_dates=True)
14     df.columns = ["rate"]
15     df["rate"] = pd.to_numeric(df["rate"], errors="coerce") # FRED uses
16     # , ,
17     df = df.loc[start:end].dropna()
18     return df["rate"] / 100 # percent -> decimal
19
20 def calibrate_vasicek(rates: pd.Series, dt: float = 1/365) -> dict:
21     """OLS calibration of Vasicek parameters via the exact AR(1) form."""
22
23     r = rates.values
24     X, Y = r[:-1], r[1:] # r_t and r_{t+dt}
25     alpha, beta, *_ = stats.linregress(X, Y)
26     s2 = np.var(Y - (alpha * X + beta), ddof=2)
27     if not 0 < alpha < 1:
28         raise ValueError(f"alpha = {alpha:.6f}: no mean reversion in
29                             this window.")
30     a = -np.log(alpha) / dt
31     b = beta / (1 - alpha)
32     sigma = np.sqrt(2 * a * s2 / (1 - alpha**2))
33     return {"a": a, "b": b, "sigma": sigma}
34
35 def mc_zcb_price(r0: float, T: float, a: float, b: float, sigma: float,
36                 n_paths: int = 50_000, n_steps: int = 1260,
37                 seed: int | None = 42) -> dict:
38     """
39     Monte Carlo ZCB price P(0,T) under Vasicek with antithetic variates.
40     n_paths antithetic pairs are simulated (2 * n_paths paths in total).
41     Returns dict: price, std_err, ci_lower, ci_upper.
42     """
43     rng = np.random.default_rng(seed)
44     dt = T / n_steps
45     e_adt = np.exp(-a * dt) # alpha
46     drift = b * (1 - e_adt) # beta
47     std = sigma * np.sqrt((1 - np.exp(-2 * a * dt)) / (2 * a)) # s
48
49     r_plus = np.full(n_paths, r0)
50     r_minus = np.full(n_paths, r0)
51     int_plus = np.zeros(n_paths) # integral of r along each path
52     int_minus = np.zeros(n_paths)
53
54     for _ in range(n_steps):
55         Z = rng.standard_normal(n_paths)
56         r_plus_new = r_plus * e_adt + drift + std * Z
57         r_minus_new = r_minus * e_adt + drift - std * Z # antithetic
58         # path
59         int_plus += 0.5 * (r_plus + r_plus_new) * dt # trapezoidal

```

```

58         rule
59         int_minus += 0.5 * (r_minus + r_minus_new) * dt
60         r_plus, r_minus = r_plus_new, r_minus_new
61
62     estimates = 0.5 * (np.exp(-int_plus) + np.exp(-int_minus))
63
64     price = estimates.mean()
65     se     = estimates.std(ddof=1) / np.sqrt(n_paths)
66     return {
67         "price": price,
68         "std_err": se,
69         "ci_lower": price - 1.96 * se,
70         "ci_upper": price + 1.96 * se,
71     }
72
73 def analytical_zcb_price(r0: float, T: float, a: float, b: float,
74                         sigma: float) -> float:
75     """Closed-form Vasicek ZCB price, for comparison."""
76     B = (1 - np.exp(-a * T)) / a
77     lnA = (b - sigma**2 / (2 * a**2)) * (B - T) - sigma**2 * B**2 / (4 *
78         a)
79     return np.exp(lnA - B * r0)
80
81 if __name__ == "__main__":
82     params = calibrate_vasicek(fetch_fed_rates()) # part (a)
83     a, b, sigma = params["a"], params["b"], params["sigma"]
84
85     r0, T = 0.04, 5.0 # given in part (c)
86     mc = mc_zcb_price(r0, T, a, b, sigma)
87     exact = analytical_zcb_price(r0, T, a, b, sigma)
88
89     print(f"a = {a:.4f}, b = {b:.4%}, sigma = {sigma:.4%}")
90     print(f"Analytical price : {exact:.6f}")
91     print(f"Monte Carlo price: {mc['price']:.6f} "
92         f"(s.e. {mc['std_err']:.2e}, 95% CI "
93         f"[{mc['ci_lower']:.6f}, {mc['ci_upper']:.6f}])")

```

Listing 3: Monte Carlo ZCB pricing with antithetic variates (monte_carlo.py)

For details and source code, see 03_monte_carlo_validation.ipynb.

Analytical Z(0,5)	Monte Carlo Z(0,5)
0.849201	0.849210

Table 3: ZCB(0,5) from analytical scheme and simulation.

2.3.2 Variance Reduction: Antithetic Variates

The standard error of plain Monte Carlo decreases only as $1/\sqrt{M}$, so halving the error requires four times as many paths. Variance reduction techniques lower the error for a fixed number of paths. Common techniques include:

- **Antithetic variates:** pair each path with its mirror image.

- **Control variates:** correct the estimate using a related quantity with a known exact value.
- **Importance sampling:** sample more often from regions that contribute most to the price.
- **Stratified sampling:** divide the random inputs into ranges and sample each range evenly.
- **Moment matching:** rescale the random numbers to have sample mean 0 and variance 1 exactly.
- **Quasi-Monte Carlo:** replace random numbers by low-discrepancy sequences (e.g. Sobol).
- **Conditional Monte Carlo:** compute part of the expectation analytically and simulate only the remainder.

Antithetic variates. For each sequence of standard normal shocks $W = (W_1, \dots, W_N) \sim \mathcal{N}(0, I_N)$ used to simulate a primary trajectory, a corresponding antithetic path is constructed using the negated sequence $-W = (-W_1, \dots, -W_N)$. If one path drifts upwards, its mirror drifts downwards by the same amount, so their errors tend to cancel. The estimator averages the two discount factors (D) in each pair:

$$\hat{P}_{AV} = \frac{2}{M} \sum_{i=1}^{M/2} \frac{D(W^{(i)}) + D(-W^{(i)})}{2}. \quad (20)$$

Since $-W$ is also standard normal, both paths have the correct distribution and the estimator remains unbiased. For one pair,

$$\text{Var}\left(\frac{D(W) + D(-W)}{2}\right) = \frac{1}{2} \left[\text{Var}(W) + \text{Cov}(D(W), D(-W)) \right]. \quad (21)$$

The discount factor decreases monotonically as the shocks push rates up, so $D(W)$ and $D(-W)$ are negatively correlated; the covariance term is negative, and the variance falls.

In our simulations, antithetic variates reduced the variance by about three orders of magnitude for the same number of paths. The gain is large because, for small σ , the discount factor is nearly linear in the shocks, and antithetic pairing cancels this linear part of the error almost completely. The method requires no additional random numbers and only a minor change to the code.

- (d) Calculate the swap rate (fixed rate leg) of an interest rate swap that has a maturity of 5 years and the floating leg has an annual coupon payment frequency linked to SOFR. **[Hint:]** Note the price of a swap is given as:

$$\text{Swap Rate} = \frac{1 - Z(N)}{\sum_{i=1}^N Z(i)}$$

Where $Z(i)$ is the zero-coupon price for maturity i .

2.4 Swap Rate (Part Id)

In an interest rate swap, one party pays a fixed rate S , and the other pays a floating rate linked to SOFR. The swap rate is the fixed rate S that makes the swap worth zero at the start, so neither party pays anything to enter it. The fixed rate S of a vanilla interest rate swap with annual payment frequency and maturity N years satisfies:

$$S = \frac{1 - Z(0, N)}{\sum_{i=1}^N Z(0, i)}. \quad (22)$$

Each $P(i)$ comes from the analytical Vasicek formula of part (b), with $r(0) = 0.04$ and the parameters a, b, σ fitted in part (a):

$$Z(i) = A(0, i) e^{-B(0, i) r(0)}, \quad i = 1, \dots, 5$$

Table 4: Illustrative ZCB prices $Z(0, T)$ for $r(0) = 0.04$ and indicative parameters $a = 0.190$, $b = 0.020$, $\sigma = 0.012$ and $\sigma = 1.2\%$.

Maturity i (yr)	ZCB price $Z(i)$
1	0.962500
2	0.929386
3	0.899871
4	0.873315
5	0.849201
$\sum Z(i)$	4.514274

Using Eq. 22 and Table 4:

$$S = \frac{1 - Z(0, 5)}{Z(0, 1) + Z(0, 2) + Z(0, 3) + Z(0, 4) + Z(0, 5)} \approx \frac{0.150799}{4.514274} \approx 3.3405\%.$$

For more details, see `02_term_structure.ipynb`.

- (e) Calculate the price of a European Call option on 5-year zero-coupon bond (with a face value of \$1000) and a maturity of 4 years and a strike price of \$900 using the Vasicek model that has been calibrated as above.

Note this solution allows for simulation of negative interest rates that is a known limitation of Vasicek model.

[**Hint:**] The payoff of call option is:

$$\max(Z[t, T] - K, 0)$$

and you may ignore the discounting of payoff or use the 4-year zero spot rate to discount the payoff.

And also note that $Z[t = 4y, T = 5y]$ can be computed using exponential affine formula where the $t = 4y$ and $r(t = 4y)$ can be simulated using same steps as Part b above.

2.5 European Call Option on a ZCB (Part Ie)

We price a European call option with:

- Underlying: 5-year ZCB with face value \$1,000
- Option maturity: $t = 4$ years
- Strike: $K = \$900$

The payoff at $t = 4$ is:

$$C(4) = \max(Z(4, 5) - 900, 0). \quad (23)$$

Estimation of the bond price at year 4. At year 4, the bond has one year left, with today's short rate $r(0) = 0.04$ and Vasicek parameters a , b , σ from part I(a). Its price depends only on the interest rate at that time, $r(4)$, through the exponential-affine formula:

$$Z(4, 5) = 1000 A(4, 5) e^{-B(4, 5) r(4)}, \quad B(4, 5) = \frac{1 - e^{-a}}{a}.$$

Calculation. We simulate 100,000 short-rate paths up to year 4 and record $r(4)$ on each path. On each path, the bond (one year left to maturity) is priced as

$$Z(4, 5) = 1000 A(4, 5) e^{-B(4, 5) r(4)}, \quad (24)$$

and the option payoff is $\max(Z(4, 5) - 900, 0)$. For details of calculations and the source code, see `03_monte_carlo_validation.ipynb`

Table 5: European call option on a ZCB ($t = 4$, $T = 5$, $K = \$900$).

Method	Option price (\$)	Std. error (\$)
Undiscounted payoff (at $t = 4$)	71.8542	0.0008
Payoff discounted with $Z(0, 4)$	62.7513	0.0007
Pathwise Monte Carlo	63.2217	0.0044
Closed-form (Vasicek)	63.2172	—

Discussion.

- The average payoff at $t = 4$ is \$71.85. Discounting it with today's 4-year ZCB price, $Z(0, 4) = 0.873315$, gives

$$C(0) = 0.873315 \times 71.8542 = \$62.75.$$

- Discounting each path with its own discount factor, $\exp(-\int_0^4 r(s) ds)$, gives \$63.22, which matches the closed-form value of \$63.22. The fixed-discount method is about \$0.47 lower because it ignores the link between payoff and discounting: paths with low rates have both a larger payoff and a larger discount factor.
- The option ends in the money on 100% of the simulated paths. Since it is always exercised, its value equals that of a forward contract on the bond,

$$1000 Z(0, 5) - 900 Z(0, 4) = \$63.22.$$

Value a European Swaption that gives the holder right to enter into a 5-year swap after 2 years. The swap rate fixed is 4.5% and will receive floating rate. Cash flows on swap are going to be exchanged semi-annually. The volatility of forward swap rate is 15%. Notional is \$100 and compounding is continuous.

- (a) Prepare the yield curve from SOFR rate for discounting. Use linear interpolation approach to get the yield curve.
- (b) Use the discount curve of (a) to value the European Swaption as mentioned.

3 Swaption Pricing (Part II)

3.1 SOFR (Secured Overnight Financing Rate) Yield Curve (Part IIa)

SOFR is the interest rate banks pay to borrow US dollars overnight, using US government bonds as security. It is the main benchmark rate for US dollar loans and derivatives. A yield curve gives the interest rate for each length of borrowing (1 month, 1 year, 5 years, and so on), and from it we obtain the discount factor $P(0, T)$: the value today of \$1 paid at time T . These discount factors are needed to value the swaption cash flows, which fall every six months between $T = 2.5$ and $T = 7$ years.

The curve is built from SOFR market data as of 1 October 2026 (Table 6). The short end uses CME Term SOFR rates and the long end uses SOFR overnight-indexed swap (OIS) rates.

Table 6: SOFR market data, 1 October 2026.

Tenor	1M	3M	6M	1Y	2Y	3Y	5Y	7Y	10Y
Rate (%)	3.9163	4.0473	4.2258	4.5446	4.59	4.68	4.71	4.75	4.83
Source	CME Term SOFR				SOFR OIS swap rates				

Short end ($T \leq 1$). CME Term SOFR rates $R(T)$ are simple rates on an ACT/360 basis. They are converted to continuously compounded zero rates by

$$y(T) = \frac{1}{T} \ln \left(1 + R(T) \frac{365 T}{360} \right). \quad (25)$$

Long end ($T \geq 2$). SOFR swap rates S_n are par rates for swaps with annual fixed payments, so they cannot be used directly as zero rates. A par swap has zero value at the start, $S_n \sum_{i=1}^n P(0, i) = 1 - P(0, n)$, which allows the discount factors to be found one year at a time (bootstrapping):

$$P(0, n) = \frac{1 - S_n \sum_{i=1}^{n-1} P(0, i)}{1 + S_n}, \quad y(n) = -\frac{\ln P(0, n)}{n}. \quad (26)$$

Swap rates at unquoted maturities (4Y, 6Y, 8Y, 9Y) are obtained by linear interpolation, and $P(0, 1)$ is taken from the 1Y Term SOFR rate.

Linear interpolation. Zero rates at any other maturity are obtained by linear interpolation between neighbouring nodes,

$$y(T) = y(T_i) + \frac{T - T_i}{T_{i+1} - T_i} [y(T_{i+1}) - y(T_i)], \quad T \in [T_i, T_{i+1}], \quad (27)$$

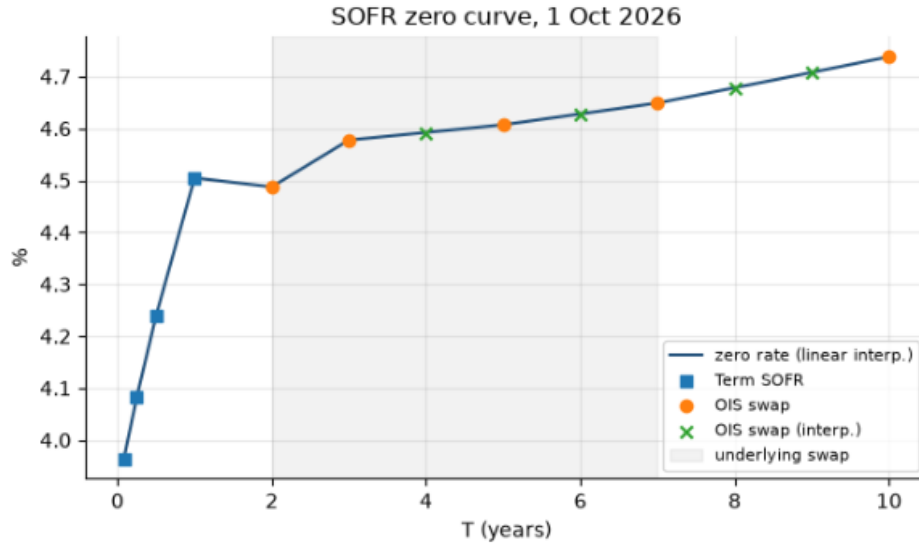


Figure 1: SOFR zero curve, 1 October 2026, built by linear interpolation of zero rates from CME Term SOFR (squares) and bootstrapped SOFR OIS swap rates (circles: quoted; crosses: interpolated). The shaded region marks the underlying swap period (2–7 years).

and discount factors follow from

$$P(0, T) = e^{-y(T)T}. \quad (28)$$

Result. The resulting zero curve is shown in Figure 1, and the discount factors on the swaption payment dates are listed in Table 7. The curve slopes gently upwards, from about 3.96% at 1 month to 4.74% at 10 years. Over the life of the underlying swap (2 to 7 years), it is nearly flat, between 4.49% and 4.65%. The small dip between 1Y and 2Y occurs where the two data sources meet, since the 1Y Term SOFR rate is slightly above the rate implied by the swap market. For details and source code, see `04_swaption_pricing.ipynb`.

Table 7: SOFR zero rates and discount factors on the swap payment grid.

T (years)	$y(T)$	$P(0, T)$	T (years)	$y(T)$	$P(0, T)$
2.0	4.4874%	0.914162	5.0	4.6070%	0.794255
2.5	4.5325%	0.892872	5.5	4.6174%	0.775726
3.0	4.5776%	0.871684	6.0	4.6278%	0.757550
3.5	4.5848%	0.851744	6.5	4.6384%	0.739712
4.0	4.5920%	0.832201	7.0	4.6490%	0.722219
4.5	4.5995%	0.813037			

Assumptions. Swap payments are taken as exactly one year apart (no day-count adjustment), and a single curve is used for both discounting and projecting SOFR.

```

1 import numpy as np
2
3 # Market data, 1 Oct 2026
4 TERM_SOFR = {1/12: 0.0391628, 3/12: 0.0404730,
5               6/12: 0.0422578, 1.0: 0.0454462} # CME Term SOFR

```

```

6 SWAP_RATES = {2.0: 0.0459, 3.0: 0.0468, 5.0: 0.0471,
7               7.0: 0.0475, 10.0: 0.0483}           # SOFR OIS swap rates
8
9
10 def build_sofr_curve(term_sofr: dict, swap_rates: dict) -> callable:
11     """
12     Build a SOFR discount curve.
13     Short end: Term SOFR (simple, ACT/360) -> zero rates.
14     Long end: swap rates bootstrapped with annual payments.
15     Zero rates are linearly interpolated.
16     Returns: function discount(T) -> P(0,T)
17     """
18     zero = {}
19
20     # Short end: simple rate -> continuously compounded zero rate
21     for T, R in term_sofr.items():
22         zero[T] = np.log(1 + R * 365 * T / 360) / T
23
24     # Long end: bootstrap annual par swap rates
25     years = np.arange(1, int(max(swap_rates)) + 1)
26     quoted = sorted(swap_rates)
27     S = np.interp(years, quoted, [swap_rates[t] for t in quoted])
28     P = {1: np.exp(-zero[1.0])}
29     for n, S_n in zip(years[1:], S[1:]):
30         annuity = sum(P[i] for i in range(1, n))
31         P[n] = (1 - S_n * annuity) / (1 + S_n)
32         zero[float(n)] = -np.log(P[n]) / n
33
34     tenors = np.array(sorted(zero))
35     rates = np.array([zero[t] for t in tenors])
36
37     def discount(T):
38         y = np.interp(T, tenors, rates)           # linear interpolation
39         return np.exp(-y * np.asarray(T))
40
41     return discount
42
43
44 discount = build_sofr_curve(TERM_SOFR, SWAP_RATES)

```

Listing 4: SOFR curve construction (curve.py)

3.2 European Swaption Valuation (Part IIb)

3.2.1 Swaption Specification

The holder has the right, in 2 years, to enter a 5-year swap where they *pay* a fixed rate of 4.5% and *receive* the floating rate. This is a **payer swaption**.

- Option expiry: $T_0 = 2$ years
- Underlying swap: 5 years, from $T_0 = 2$ to $T_n = 7$ years
- Fixed rate (strike): $K = 4.5\%$
- Payments: semi-annual, so $\tau = 0.5$ and $T_i = T_0 + 0.5i$ for $i = 1, \dots, 10$
- Notional: $N = \$100$
- Volatility of the forward swap rate: $\nu = 15\%$
- Discounting: SOFR curve from part (a), continuous compounding

3.2.2 Annuity Factor and Forward Swap Rate

The **annuity factor** is the present value of \$1 per year, paid on each fixed-leg date:

$$A = \sum_{i=1}^n \tau_i P(0, T_i), \quad (29)$$

with $n = 10$ and $\tau_i = 0.5$.

The **forward swap rate** is the fixed rate that makes the forward swap worth zero today:

$$S_0 = \frac{P(0, T_0) - P(0, T_n)}{A}. \quad (30)$$

The discount factors needed from the curve are:

T	$r(T)$	$P(0, T)$
2.0	4.4922%	0.914074
2.5	4.5365%	0.892782
3.0	4.5809%	0.871599
3.5	4.5851%	0.851736
4.0	4.5894%	0.832290
4.5	4.5993%	0.813046
5.0	4.6092%	0.794169
5.5	4.6194%	0.775641
6.0	4.6296%	0.757467
6.5	4.6401%	0.739631
7.0	4.6505%	0.722140

Table 8: Interpolated discount factors on the swaption dates.

3.2.3 Black's Formula

At expiry, the payer swaption pays

$$V(T_0) = N \cdot A(T_0) \cdot \max(S(T_0) - K, 0). \quad (31)$$

We use the annuity $A(t)$ as the numéraire. Under the annuity measure $\mathbb{A}$ (Section 3.3), the forward swap rate is a martingale. We assume it is lognormal:

$$dS(t) = \nu S(t) dW^{\mathbb{A}}(t). \quad (32)$$

This gives Black's formula:

$$V_{\text{swaption}} = N \cdot A \cdot [S_0 \Phi(d_+) - K \Phi(d_-)], \quad (33)$$

where

$$d_+ = \frac{\ln(S_0/K) + \frac{1}{2}\nu^2 T_0}{\nu\sqrt{T_0}}, \quad (34)$$

$$d_- = d_+ - \nu\sqrt{T_0}. \quad (35)$$

Quantity	Value
Annuity A	4.025250
Forward swap rate S_0	4.7683%
d_+	0.379026
d_-	0.166894
$\Phi(d_+)$	0.647666
$\Phi(d_-)$	0.566273
Payer swaption value V	\$2.1737

Table 9: Payer swaption valuation with the SOFR curve.

3.2.4 Result

For details and source code, see 04.swaption.pricing.ipynb. **Comments.**

- The forward swap rate (4.768%) is above the strike (4.5%). So the swaption is *in the money*: paying 4.5% fixed is cheaper than the market rate.
- The intrinsic value is $NA(S_0 - K) = 100 \times 4.0253 \times 0.002683 = \1.08 . The rest, about \$1.09, is time value from volatility.
- The forward swap rate is close to the 7-year par rate (4.75%). This makes sense, because the curve is flat between 2 and 7 years.

3.2.5 Implementation

```

1 import numpy as np
2 from scipy.stats import norm
3
4 deposits = {1/12: 0.0391, 0.25: 0.0402}
5 swaps = {1: 0.0439, 2: 0.0459, 3: 0.0468, 4: 0.0469,
6          5: 0.0471, 7: 0.0475, 10: 0.0483}
7
8 # ---- (a) bootstrap + linear interpolation of zero rates
9 def build_curve(deposits, swaps, max_year=10):
10     T, Z = [], []
11     for t, r in deposits.items():
12         P = 1.0 / (1.0 + r * t)
13         T.append(t); Z.append(-np.log(P) / t)
14
15     ten, rate = np.array(list(swaps)), np.array(list(swaps.values()))
16     P = {}
17     for k in range(1, max_year + 1):
18         S = np.interp(k, ten, rate) # fills 6Y, 8Y, 9Y
19         P[k] = (1 - S * sum(P[j] for j in range(1, k))) / (1 + S)
20         T.append(float(k)); Z.append(-np.log(P[k]) / k)
21     return np.array(T), np.array(Z)
22
23 T_pil, Z_pil = build_curve(deposits, swaps)
24 zero_rate = lambda t: np.interp(t, T_pil, Z_pil)
25 discount = lambda t: np.exp(-zero_rate(t) * t)
26
27 # ---- (b) Black's formula
28 def european_swaption(notional, K, T0, swap_tenor, freq, vol,
29                       discount, payer=True):
30     n = int(round(swap_tenor / freq))
31     times = [T0 + (i + 1) * freq for i in range(n)]

```

```
32
33     A = sum(freq * discount(t) for t in times)
34     S0 = (discount(T0) - discount(times[-1])) / A
35
36     sd = vol * np.sqrt(T0)
37     d_plus = (np.log(S0 / K) + 0.5 * sd**2) / sd
38     d_minus = d_plus - sd
39
40     if payer:
41         return notional * A * (S0 * norm.cdf(d_plus)
42                                - K * norm.cdf(d_minus))
43     return notional * A * (K * norm.cdf(-d_minus)
44                            - S0 * norm.cdf(-d_plus))
45
46 V = european_swaption(100, 0.045, 2.0, 5.0, 0.5, 0.15, discount)
47 print(f"Payer swaption value: {V:.4f}") # 2.1737
```

Listing 5: SOFR curve and swaption pricing (swaptions.py)

- (c) Explain the concept of numeraire and suggest a suitable numeraire to solve the valuation of a European Swaption. (d) Suggest a similar numeraire for a Bermudan swaption and explain the valuation.

3.3 Numeraire Theory (Part IIc and IId)

3.3.1 The Concept of Numeraire

A **numeraire** is a strictly positive asset $N(t)$ used to denominate prices. By the fundamental theorem of asset pricing, for any numeraire $N(t)$ there exists an associated equivalent martingale measure $\mathbb{Q}^N$ under which the price of any tradeable asset, expressed in units of N , is a martingale:

$$\frac{V(t)}{N(t)} = \mathbb{E}^{\mathbb{Q}^N} \left[\frac{V(T)}{N(T)} \mid \mathcal{F}_t \right]. \quad (36)$$

The choice of numeraire is a modelling tool: different numeraires simplify the pricing of different instruments.

3.3.2 Numeraire for the European Swaption

The natural numeraire for a European swaption is the **annuity** $A(t) = \sum_i \tau_i P(t, T_i)$. Under the associated **swap measure** $\mathbb{Q}^A$, the forward swap rate $S(t)$ is a martingale:

$$S(t) = \mathbb{E}^{\mathbb{Q}^A} [S(T_0) \mid \mathcal{F}_t]. \quad (37)$$

Assuming log-normality of $S(t)$ under $\mathbb{Q}^A$ with volatility ν , Black's formula (33) follows directly. This is the theoretical justification for Black's model in swaption markets.

3.3.3 Numeraire for the Bermudan Swaption

A Bermudan swaption grants the holder the right to enter the swap on any of M exercise dates $\{T_0, T_1, \dots, T_{M-1}\}$. Multiple exercise dates preclude the use of a single annuity numeraire, because the post-exercise annuity changes with each exercise time.

The standard choice is the **bank account** (savings account) $B(t) = \exp\left(\int_0^t r(s) ds\right)$, corresponding to the risk-neutral measure $\mathbb{Q}$. Under $\mathbb{Q}$, the value of the swaption satisfies the recursive backward optimality condition:

$$V(t, r) = \max \left(\text{intrinsic}(t, r), \mathbb{E}^{\mathbb{Q}} \left[e^{-\int_t^{t+1} r(s) ds} V(t+1, r) \mid r(t) \right] \right). \quad (38)$$

Longstaff-Schwartz (LSM) regression approximates the continuation value using a regression of discounted future cash flows on basis functions of the current state $r(t)$. This avoids explicit backward induction on a grid while remaining computationally tractable for high-dimensional state spaces.

Numeraire summary:

- European swaption → **Annuity measure** $\mathbb{Q}^A$; forward swap rate is a martingale; Black's formula applies.
- Bermudan swaption → **Risk-neutral measure** $\mathbb{Q}$ (bank account); backward

optimisation required; LSM regression is the standard numerical method.

4 Code Repository Structure

All calculations are done in Python. The full code, written to production-level standards, is available in the project repository at https://github.com/QuantHL/IIQF_Project_Report/tree/main. The code in the repository is organised differently from the shorter code listings shown in this report, which are simplified to explain each step.

```

1 |
2 |-- data/                                # Raw FRED / SOFR CSV pulls (timestamped)
3 |-- src/
4 |   |-- vasicek.py                       # ZCB pricing:  $B(T)$ ,  $A(T)$ ,  $zcb\_price()$ 
5 |   |-- calibration.py                   # OLS calibration from FRED data
6 |   |-- curve.py                         # SOFR yield curve interpolation
7 |   |-- monte_carlo.py                   # MC simulation + antithetic variates
8 |   |-- swaps.py                         # Swap rate from ZCB strip
9 |   |-- swaptions.py                     # Black's formula for swaptions
10 |-- notebooks/
11 |   |-- 01_calibration.ipynb
12 |   |-- 02_term_structure.ipynb
13 |   |-- 03_monte_carlo_validation.ipynb
14 |   |-- 04_swaption_pricing.ipynb
15 |-- tests/                               # Unit tests (analytical vs MC agreement)
16 |-- requirements.txt
17 '-- README.md

```

Listing 6: Project directory layout